# Web Development Fundamentals

## Session 01 — Web Concepts & Mental Models

*How the Web works — No coding yet, just understanding!*

**M. Souhil** | Dev Web Core Team

**Data Science Club — USTHB**

Tech Department | Dev Web Section

March 6, 2026 | Session 01 / 07

# Today's Journey 🗺

1. **Hook** — Why this matters
2. **Internet vs Web** — Not the same!
3. **URLs, Domains, DNS** — @ system
4. **How the Web Works** — Req & Res
5. **Browser's Role** — More than a window
6. **Web Languages** — HTML, CSS, JS
7. **Static vs Dynamic** — Two worlds
8. **Dev Tools** — Your toolkit
9. **Recap** — Key mental models
10. **Mini-Challenge** — Let's play!

**Note**

No coding today — we are building **mental models**. Code comes next session!

# Hook — Why Web Development?

–

# Every Day, You Use It Without Thinking

Right now, at this very moment on Earth:

- **5.5 billion** people are online
- **250,000** new websites go live *every day*
- **6 trillion** web pages exist and are indexed
- **90%** of human knowledge lives on the Web
- Every app, every service, every dashboard — powered by web tech

**The Web is the world's largest information system.**

**You are about to learn how to build it.**

# What Will You Be Able to Do After This Training?

## After This Session

- Explain how the web works to anyone
- Read a URL like a professional
- Know what HTML, CSS & JS each do
- Use browser DevTools confidently
- Understand what happens behind a click

## After the Full Training

- Build & deploy a real website
- Create interactive pages with JavaScript
- Connect front-end to back-end
- Work with APIs and databases
- Land your first web dev project

**Analogy**

**Analogy:** Think of today as learning to read a city map before driving. You will not drive yet — but once you understand the streets, everything else becomes easy.

# Internet vs The Web

# The #1 Confusion in Tech ⚠

## Internet $\neq$ Web

### The Internet

A **global physical network**

Cables, routers, satellites, fiber

Connects billions of devices

Exists since the **1960s** (ARPANET)

Carries *all* kinds of data traffic

Email, file transfer, video calls, web — all run ON the Internet

### The Web (WWW)

A **service** that runs on the Internet

Made of linked pages and resources

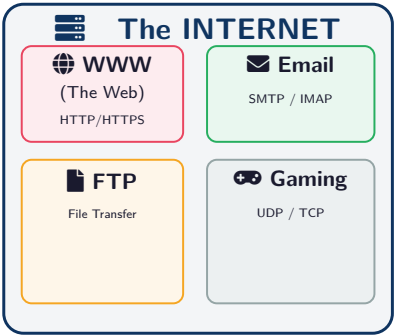
Uses HTTP/HTTPS protocol

Invented by **Tim Berners-Lee** in 1989

What you access via a **browser**

Just *one* of many Internet services

# Internet vs Web — The Analogy




**Analogy**

**Think of it this way:**

The **Internet** is like the entire **city road network** — highways, streets, intersections.

The **Web** is like the **postal delivery service** — it uses those roads, but so do ambulances, taxis, and buses.

**All these services share the roads but are NOT the same service!**

**Key Takeaway**

Internet = Infrastructure → Web = One Service on it

# Quick Check ?— Internet or Web?

For each item, shout: **"Internet!"** or **"Web!"**

**Q1.** You open `google.com` in Chrome

**Q2.** You send an email to someone

**Q3.** You video call a friend on Zoom

**Q4.** You browse Wikipedia

**Q5.** You download a game via Steam

Answers (reveal after discussion):

**A1.** ..... (`http://google.com`)

**A2.** ..... (SMTP protocol)

**A3.** ..... (Zoom's own protocol)

**A4.** ..... (`https://wikipedia.org`)

**A5.** ..... (Steam protocol)

**Note**

The Web always uses **HTTP or HTTPS** — if it's in the address bar, it's the Web!

# URLs, Domains, IPs & DNS

# Every Website Has an Address

**Analogy**

A URL is like a **postal address for a document**: `Protocol` = delivery method, `Domain` = the building name, `Path` = the specific room or floor inside.

# URL Deep Dive — Each Part Explained 🔍

| Part | Example | What it means |
|---|---|---|
| Protocol | `https://` | How data travels (secure vs not) |
| Subdomain | `www.` or `mail.` | Sub-section of a domain |
| Domain | `google`, `usthb` | Human-readable name for a server |
| TLD | `.com`, `.dz`, `.org` | Top-Level Domain — category/country |
| Port | `:443`, `:8080` | Door number on the server (often hidden) |
| Path | `/wiki/Algeria` | Specific page or resource |
| Query | `?q=web+dev` | Search or filter parameters |
| Fragment | `#section-2` | Jump to a specific part of the page |

```
Full example:  https://docs.mozilla.org/en-US/docs/Web/HTML#intro
```

# IP Addresses — The Real Address

Every device on the Internet has an **IP address**

IP = **Internet Protocol** address

It is a numerical label: `142.250.185.78`

**IPv4**: 4 groups of numbers (0–255) — `192.168.1.1`

**IPv6**: newer, longer format — `2001:db8::1`

Domains are just **human-friendly nicknames** for IPs

No one wants to type `142.250.185.78` instead of `google.com`!

**Famous IPs**

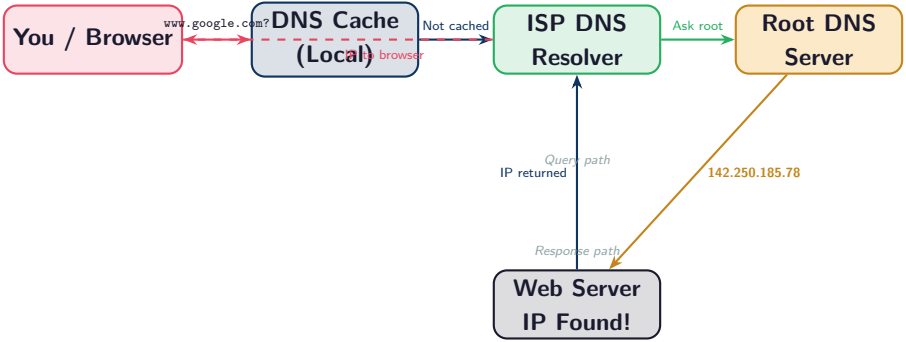
| | |
|---|---|
| `google.com` | `142.250.x.x` |
| `wikipedia.org` | `208.80.x.x` |
| `localhost` | `127.0.0.1` |
| `Your router` | `192.168.1.1` |

(x.x = varies by region)

**Note**

Try now: open your terminal and type `ping google.com` — you will see its IP address!

# DNS — The Internet's Phone Book 📖



# DNS — Step by Step

**Step 1.** **You type** `www.google.com` in your browser

**Step 2.** **Browser checks its cache** — "Do I already know this IP?"

**Step 3.** **OS checks its local cache** — same question

**Step 4.** **Query goes to your ISP's DNS Resolver** (Algérie Télécom, Djezzy, etc.)

**Step 5.** **Resolver asks a Root Name Server** — "Who handles `.com`?"

**Step 6.** **Root says:** "Ask the `.com` TLD server"

**Step 7.** **TLD server says:** "Google's authoritative server is at X"

**Step 8.** **Authoritative DNS returns:** IP address `142.250.185.78`

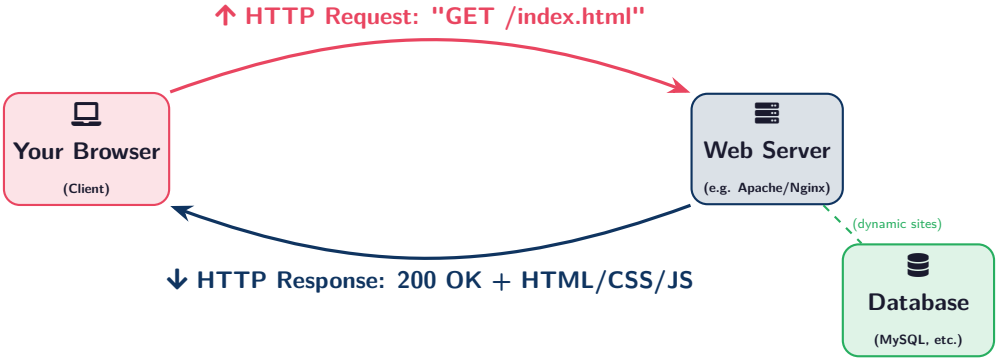
**Step 9.** **Browser connects** to that IP — you see Google's homepage

**Analogy**

DNS is like asking a **librarian**: "Where is the book called *google.com*?" The librarian checks the catalog, finds the shelf number (IP), and sends you there.

# How the Web Works

# The Request-Response Cycle




**Key Takeaway**

Every webpage load = at least one **Request** (you asking) + one **Response** (server answering)

# HTTP Status Codes — The Server's Language

## Success (2xx)

**200 OK** — Everything worked!

**201 Created** — New resource made

**204 No Content** — Done, nothing to show

## Redirects (3xx)

**301 Moved Permanently** — New URL forever

**302 Found** — Temporary redirect

**304 Not Modified** — Use cached version

## Client Errors (4xx)

**400 Bad Request** — Malformed request

**401 Unauthorized** — Login required

**403 Forbidden** — Access denied

**404 Not Found** — Page doesn't exist!

**500 Internal Error** — Server broke

**503 Unavailable** — Server is down

**Note**

**Rule:** 4xx = **your fault** (client). 5xx = **server's fault**. Remember this!

# HTTP Methods — How You Ask 👆

| Method | What it means |
|---|---|
| `GET` | Retrieve / Read data (visit a page) |
| `POST` | Send data (submit a form, login) |
| `PUT` | Update something completely |
| `PATCH` | Update something partially |
| `DELETE` | Remove a resource |

**Analogy**

HTTP methods are like **actions at a restaurant**:

**GET** = Read the menu
**POST** = Place an order
**PUT** = Change your whole order
**PATCH** = Add extra sauce only
**DELETE** = Cancel your order

**Note**

GET and POST are the two you will use 95% of the time in beginner web dev. The others matter more in APIs.

# HTTPS — Security on the Web 🔒

**HTTP** = plain text — anyone can read your data

**HTTPS** = HTTP + **TLS/SSL encryption**

Your data is **encrypted end-to-end**

Look for the 🔒 padlock in your browser bar

**Never** enter passwords on HTTP-only sites

Modern browsers **warn** you about HTTP sites

Google penalizes HTTP sites in search rankings

Certificates are now **free** via Let's Encrypt!

🔒 `https://google.com`

Connection is Secure ✓

⚠ `http://example.com`

Not Secure ✗

**Rule of Thumb**

If the padlock is missing → **Don't type passwords or card details!**

# The Browser's Role

# Your Browser is a Supercomputer

The browser is NOT just a window to the internet.
It is a **full application runtime** that:

- **Downloads** HTML, CSS, JS, images
- **Parses & renders** the HTML structure (DOM)
- **Applies** CSS styling rules
- **Executes** JavaScript code
- **Manages** network requests
- **Enforces** security policies
- **Caches** resources for speed
- **Exposes** Developer Tools to you

**Browser Engines**

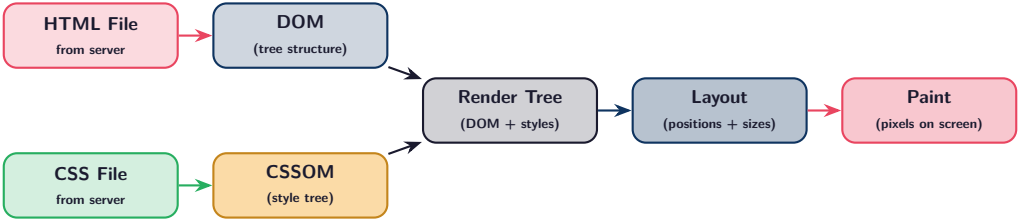
| | |
|---|---|
| Chrome / Edge | Blink |
| Firefox | Gecko |
| Safari | WebKit |

Same HTML code renders slightly differently in each engine!

**Analogy**

The browser is like a **universal translator** — it reads code and shows you a visual page.

# How the Browser Renders a Page




**DOM** = the HTML turned into a **tree of objects** the browser can work with

**CSSOM** = the CSS turned into a **style map** for each element

**Render Tree** = combines both to know what to draw

**Layout** = calculates where each element goes on screen

**Paint** = actually draws pixels to your screen

# Browser Developer Tools — Your Secret Weapon 🔧

**How to open DevTools:**

Windows / Linux: **F12** or **Ctrl+Shift+I**

macOS: **Cmd+Option+I**

Right-click → **Inspect**

**Key tabs you'll use:**

- **Elements** — Inspect & edit HTML/CSS live
- **Console** — Run JS, see errors
- **Network** — See all requests & responses
- **Application** — Cookies, localStorage
- **Performance** — Page speed analysis
- **Device Mode** — Test mobile layouts

**Live Demo Tip**

Right now: open any website, press **F12**, go to **Elements**, double-click any text, and change it!

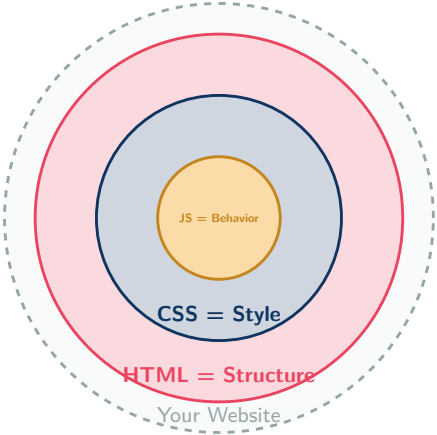
This does NOT change the real website — only your local view.

**Note**

DevTools is free, built-in, and used by every web developer on Earth — every single day.

# Web Languages Basics

# The Three Languages of the Web




**HTML**
The skeleton — what exists on the page

**CSS**
The skin — how it looks

**JavaScript**
The muscles — what it does

# HTML — The Structure

HTML = **HyperText Markup Language**

Describes the **structure and content** of a page

Uses **tags**: `<tag>content</tag>`

Every HTML file begins with `<!DOCTYPE html>`

Organized as a **tree structure** (parent/child elements)

This tree is called the **DOM** (Document Object Model)

HTML is **NOT** a programming language — it is a markup language

Has **no logic, no loops, no conditions** — just structure

**Basic HTML Example**

```
<!DOCTYPE html> <html> <head>
<title>My Page</title> </head>
<body> <h1>Hello, World!</h1>
<p>Welcome to USTHB DSC.</p> <a
href="https://usthb.dz"> Visit USTHB
</a> <img src="logo.png" alt="logo">
</body> </html>
```

**Analogy**

HTML is like the **skeleton** of a human body — it gives shape and holds everything in place, but alone it looks bare.

# Common HTML Elements — Just a Sample

| Tag | Purpose | Tag | Purpose |
|---|---|---|---|
| `<h1> to <h6>` | Headings (big to small) | `<form>` | User input form |
| `<p>` | Paragraph of text | `<input>` | Text field, checkbox, etc. |
| `<a>` | Link to another page | `<button>` | Clickable button |
| `<img>` | Display an image | `<table>` | Data table |
| `<ul> / <li>` | Unordered list / list item | `<header>` | Page/section header |
| `<div>` | Generic container block | `<nav>` | Navigation links |
| `<span>` | Inline container | `<footer>` | Footer area |

**Note**

There are **100+** HTML elements, but 20–30 cover 95% of real-world use. Learn those first.

# CSS — The Stylist Alt

CSS = **Cascading Style Sheets**
Controls **how HTML elements look**
Colors, fonts, sizes, spacing, layout, animations
Uses **selectors** to target HTML elements
The **"Cascade"** = rules applied in priority order
Has a **Box Model**: every element is a box (margin, border, padding, content)
**Flexbox** and **Grid** are the two main layout systems
CSS is **not** a programming language — it is a style language

**CSS Example**

```
/* Select by tag */ h1  color:
#E94560; font-size:  32px;
/* Select by class */ .card
background:  white; border-radius:
8px; padding:  16px; box-shadow:
0 2px 8px gray;
/* Select by ID */ #navbar
position:  fixed; top:  0; width:
100
```

**Analogy**

CSS is like the **interior designer** — same room (HTML), but they choose the paint, furniture, and lights.

# JavaScript — The Brain JS

JavaScript = the **only native programming language** of browsers

Adds **interactivity and behavior** to pages

Can **read and modify** the HTML (DOM) and CSS

Can **send/receive data** from servers (Fetch API)

Runs **in your browser** — client-side by default

Also runs **on servers** via **Node.js**

Powers **React, Vue, Angular** frameworks

**Most used language on GitHub** for 12+ years

### JS Example

```
// Change page content
document.querySelector("h1") .textContent =
"Hello USTHB!";
// Respond to a click
button.addEventListener("click", function()
alert("You clicked me!");  );
// Fetch data from an API
fetch("https://api.example.com/data") .then(res
=> res.json()) .then(data => console.log(data));
```

### Analogy

JS is like the **electrical system** of a building — invisible until you flip a switch.

# HTML + CSS + JS Together — The Power Combo

# Static vs Dynamic Websites

# Two Types of Websites ⚖️

## Static Website

- Pre-built HTML/CSS/JS files
- **Same content** for every visitor
- No database needed
- **Very fast** to load — just files
- Easy and cheap to host
- Changes require **editing code files**
- Great for: portfolios, docs, landing pages

**Examples:** Wikipedia (mostly), GitHub Pages sites, documentation pages, marketing pages

## Dynamic Website

- Content **generated on the fly**
- Content changes per visitor/request
- Requires a **database** and back-end
- More complex infrastructure
- Enables **user accounts, personalization**
- Content managed via **CMS or API**
- Great for: social media, e-commerce, dashboards

**Examples:** Facebook, Instagram, Amazon, Gmail, YouTube

# Static vs Dynamic — Side-by-Side Comparison

| | Static | Dynamic |
|---|---|---|
| **Content** | Same for all visitors | Personalized per user |
| **Back-end?** | No | Yes |
| **Database?** | No | Yes (usually) |
| **Speed** | Very fast | Depends on server |
| **Hosting cost** | Low or free | Higher |
| **Complexity** | Low | High |
| **Login system?** | No | Yes |
| **Updates** | Edit files manually | Via admin panel/CMS |
| **Technologies** | HTML, CSS, JS | + PHP, Python, Node.js, SQL |
| **Best for** | Portfolios, docs | Apps, platforms, shops |

**Key Takeaway**

This training starts with **Static** — master it first, then layer in Dynamic. One step at a time!

# The Full Web Architecture Picture

**FRONT-END**

HTML + CSS + JavaScript (runs in the Browser)

HTTP req/res

**BACK-END**

Node.js / Python (Flask/Django) / PHP / Ruby — runs on the Server

SQL queries

**DATABASE**

MySQL / PostgreSQL / MongoDB / Redis — data storage

**Note**

**Full-Stack Developer** = someone who can build all three layers. You are on the path!

# Basic Web Dev Tools

# Your Starter Toolkit 🛠

## Code Editors

**VS Code** ← **Recommended!** Free, powerful, universal

Sublime Text — Lightweight, fast

Brackets — Beginner-friendly

Vim / Neovim — For power users

## Browsers for Dev

**Chrome** ← Best DevTools

Firefox Developer Edition

Edge (Chromium-based)

Safari (macOS only)

## Version Control

**Git** — Track changes, collaborate

**GitHub** — Host code, portfolio

GitLab — CI/CD friendly

## Online Sandboxes

**CodePen** — HTML/CSS/JS playground

**JSFiddle** — Quick JS testing

**StackBlitz** — Full projects online

**Replit** — Shareable live coding

# VS Code — Your Daily Driver

**Why VS Code?**

- Completely **free and open source**
- Built by Microsoft, trusted by millions
- **Extensions** for every language and tool
- Built-in **Git integration**
- **Live Share** for remote pair programming
- Integrated **terminal**
- **IntelliSense** — smart autocomplete
- Runs on Windows, macOS, Linux

**Download:**
`code.visualstudio.com`

## Must-Have Extensions

**Live Server** — Auto reload browser

**Prettier** — Format code automatically

**HTML CSS Support** — Autocomplete

**ESLint** — Catch JS errors early

**GitLens** — Powerful git history

**Material Icons** — Nice file icons

# Git & GitHub — Why Every Developer Uses Them

## What is Git?

A **version control system** — tracks every change

Like **Ctrl+Z on steroids**, forever, for entire projects

Lets you create **branches**: work on features without breaking main code

Enables **collaboration**: multiple devs on the same codebase

## What is GitHub?

A **cloud platform** to store git repositories

Your **coding portfolio** — future employers look here

Free for public projects + collaboration

### Basic Git Commands

```
# Start tracking a project git
init
# Save a snapshot git add .  git
commit -m "My first commit"
# Push to GitHub git remote add
origin URL git push origin main
# Get others' changes git pull
```

### Note

Start using Git from Day 1. Even for solo projects. Your future self will thank you!

# Recap & Key Takeaways

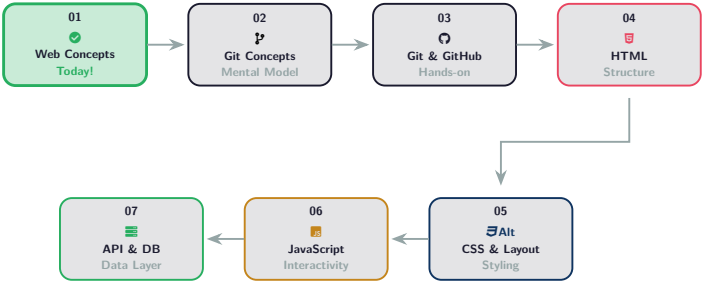
# The Mental Models You Now Own 💡

1. **Internet** $\neq$ **Web** — Infrastructure vs Service
2. **URL anatomy** — Protocol, Domain, Path
3. **DNS** = Internet's phone book: name $\rightarrow$ IP
4. **Request-Response** = fundamental web loop
5. **HTTP verbs** = GET, POST, PUT, DELETE
6. **Browser = rendering engine**, not just a viewer
7. **HTML** = structure, **CSS** = style, **JS** = behavior
8. **Static** = pre-built files; **Dynamic** = generated per request
9. **Front-End + Back-End + DB** = full web stack
10. **VS Code + Git** = your first essential tools

**🔑 Key Takeaway**

**You now understand HOW the web works.**
**Next sessions: you start BUILDING it.**

# What's Coming Next — Full Roadmap




## Before Session 02 (Git)

Install **Git** (`git-scm.com`)

Install **VS Code**

Create a **GitHub account**

Read: *"What is version control?"* (MDN)

## Resources

`developer.mozilla.org` (MDN)

`git-scm.com/book` (Git book, free)

`w3schools.com` (beginner HTML/CSS)

`javascript.info` (JS bible)

# Mini-Challenges — Let's Play!

# Let's Test Your Knowledge!

**We Have 4 Challenges:**

**Challenge 1** — Internet or Web?

**Challenge 2** — URL Breakdown Battle

**Challenge 3** — DNS Roleplay Game

Duration: 20–25 minutes total

# Challenge 1 — Internet or Web? ❓

**CHALLENGE 1: "Internet or Web?" — 5 minutes**

**Format:** Kahoot-style MCQ (verbal or via Kahoot app)
**Teams:** Individual — fastest correct answer gets a point!

**Questions:**

1) You send a tweet on Twitter. Web or Internet application?

2) You download a Linux ISO via `https://ubuntu.com` — Web or Internet?

3) You make a Skype video call — Web or Internet?

4) You use WhatsApp Web at `web.whatsapp.com` — Web or Internet?

5) You SSH into a remote server — Web or Internet?

6) You stream Netflix — Web or Internet?

**Answers**

1) **Web** (browser-based, HTTPS)

2) **Web** (download via HTTPS URL)

3) Internet (custom protocol)

4) **Web** (web.whatsapp.com = HTTPS)

5) Internet (SSH protocol)

6) Internet (RTSP/HLS streaming)

# Challenge 2 — URL Breakdown Battle

**CHALLENGE 2: URL Breakdown — 5 minutes**

**Format:** Teams of 2–3. Each team gets a URL strip to label.
**Goal:** Label each part (Protocol, Subdomain, Domain, TLD, Port, Path, Query, Fragment)
**Scoring:** 1 pt per correct label. Fastest complete set wins bonus points!

**URLs to break down:**

```
Team A: https://mail.google.com/mail/u/0/?tab=rm#inbox
Team B: https://www.youtube.com/watch?v=dQw4w9WgXcQ&list=PLx
Team C: https://en.wikipedia.org/wiki/Algeria#History
Team D: https://shop.amazon.com:443/s?k=laptop&page=2
```

**Analogy**

**Think of each URL like a full mailing address.** You wouldn't send a package with half an address — every part matters!

# Challenge 3 — DNS Roleplay Game

**CHALLENGE 3: DNS Roleplay — 7 minutes**

**Roles to assign** (1–2 students each):

**User (Browser)** **ISP DNS Resolver** **Root Name Server**
**TLD Server (.com)** **Authoritative DNS Server** **Web Server**

**Script to act out:**

1) **User:** "I want to visit `www.github.com`! Does anyone know the address?"
2) **ISP Resolver:** "I don't know it. Let me ask the Root Server."
3) **Root Server:** "I don't know GitHub, but ask the `.com` TLD server!"
4) **TLD Server (.com):** "GitHub's authoritative server is over there!"
5) **Authoritative Server:** "GitHub's IP is `140.82.114.4`!"
6) **ISP Resolver** tells **User:** "Found it! I'll remember it for next time!"
7) **User/Browser** walks to **Web Server:** "Hi! Can I get your homepage?"
8) **Web Server** hands a slip of paper: "Here's your HTML — 200 OK!"

# You're Ready.

The Web isn't magic anymore —
it's a system you now understand.

mokeddemsouhil968@gmail.com
github.com/datascienceclubusthb-club

Session 01 of 07

# Appendix — HTTP Headers Deep Dive

**HTTP Request Headers** tell the server *about the client*:

```
GET /index.html HTTP/1.1 Host: www.github.com User-Agent: Mozilla/5.0
(Chrome/120) Accept: text/html,application/xhtml+xml Accept-Language:
en-US,en;q=0.9 Connection: keep-alive Authorization: Bearer eyJhbGci...
```

**HTTP Response Headers** tell the client *about the response*:

```
HTTP/1.1 200 OK Content-Type: text/html; charset=UTF-8 Content-Length: 24682
Cache-Control: max-age=3600 Set-Cookie: session_id = abc123; HttpOnly; SecureX -
Frame - Options : DENY
```

# Appendix — Hosting Options Comparison

| Platform | Static | Dynamic | Free tier | Best for |
|---|---|---|---|---|
| **GitHub Pages** | Yes | No | Yes | Portfolios, docs |
| **Netlify** | Yes | Limited | Yes | Static, forms |
| **Vercel** | Yes | Yes | Yes | React/Next.js |
| **Railway** | No | Yes | Limited | Node, Python |
| **Render** | Yes | Yes | Yes | Full-stack |
| **Heroku** | No | Yes | Paid | Classic PaaS |
| **AWS / GCP / Azure** | Yes | Yes | Limited | Enterprise |

**Key Takeaway**

**Recommendation for this training:** Start with GitHub Pages or Netlify — zero cost, zero config, deploy in minutes. No excuses not to go live!

# Appendix — Glossary of Key Terms

| Term | Definition |
|---|---|
| HTTP | Protocol for web communication |
| HTTPS | Encrypted HTTP (TLS) |
| DNS | Domain → IP translation system |
| DOM | Browser's HTML tree structure |
| URL | Web address (full path) |
| IP | Numerical device address |
| TLD | Last part of a domain (.com) |
| CDN | Servers near users for speed |
| API | Interface for app-to-app data |

| Term | Definition |
|---|---|
| HTML | Structure markup language |
| CSS | Style / layout language |
| JS | Web programming language |
| GET | HTTP method to fetch data |
| POST | HTTP method to send data |
| 200 OK | Successful HTTP response |
| 404 | Page not found error |
| SSL/TLS | Encryption protocols |
| Git | Version control system |